

# APPROF Practical Work, Deep Learning using CNN and KNN

Vasco Silva and José Santos  
*Instituto Superior de Engenharia do Porto (ISEP)*  
1240499@isep.ipp.pt, 1232153@isep.ipp.pt

**Abstract**—This study presents a comprehensive framework for the development and evaluation of Deep Learning models applied to multi-class fruit classification using the Fruits-360 dataset. We investigate the architectural and computational trade-offs inherent in building Convolutional Neural Networks (CNNs), comparing a custom-designed scratch-built architecture against industry-standard transfer learning models, specifically ResNet50V2 and MobileNetV2. Our implementation prioritizes training efficiency through the tf.data API, utilizing prefetching and caching to optimize GPU throughput. The experimental results demonstrate that while custom architectures are valuable for theoretical understanding, transfer learning paradigms achieve superior performance, with the ResNet50V2 champion model reaching a classification accuracy of 96.84, significantly exceeding the project objective of 80. We further provide a qualitative and quantitative analysis of the network's feature extraction capabilities through activation map visualizations and per-class error distribution studies.

**Index Terms**—Deep Learning, Convolutional Neural Networks (CNN), Transfer Learning, Image Classification, Fruits-360, Performance Optimization.

## I. INTRODUCTION

Convolutional Neural Networks (CNNs) represent the state-of-the-art in computer vision, fundamentally altering how machines interpret visual data. Unlike traditional feed-forward neural networks, CNNs leverage the spatial hierarchy of images through local receptive fields, shared weights, and spatial pooling. By employing a series of convolutional layers, the network acts as an automated feature extractor, identifying low-level primitives such as edges and textures in early layers and high-level, complex semantic patterns in deeper layers. The efficacy of a CNN is governed by its architectural hyperparameters, including filter dimensions, stride configurations, padding strategies, and pooling mechanisms. These parameters dictate the network's ability to maintain spatial invariance while reducing dimensionality. In the context of this study, we explore these mechanics by implementing and comparing custom scratch-built architectures against established transfer-learning paradigms, such as ResNet50 and MobileNetV2. The challenge lies in balancing model complexity with computational efficiency to ensure high-accuracy classification while preventing overfitting. This work bridges theoretical understanding with practical implementation using Python and TensorFlow/Keras to achieve a robust multi-class classification framework

## II. LITERATURE REVIEW

## III. INTRODUCTION TO CNN

### A. CNN Theory and Project Objectives

Convolutional Neural Networks (CNNs) represent the state-of-the-art in computer vision, fundamentally altering how machines interpret visual data. Unlike traditional feed-forward neural networks, CNNs leverage the spatial hierarchy of images through local receptive fields, shared weights, and spatial pooling. By employing a series of convolutional layers, the network acts as an automated feature extractor, identifying low-level primitives such as edges and textures in early layers and high-level, complex semantic patterns in deeper layers.

The efficacy of a CNN is governed by its architectural hyperparameters, including filter dimensions, stride configurations, padding strategies, and pooling mechanisms. These parameters dictate the network's ability to maintain spatial invariance while reducing dimensionality. In the context of this study, we explore these mechanics by implementing and comparing custom scratch-built architectures against established transfer-learning paradigms, such as ResNet50 and MobileNetV2. The challenge lies in balancing model complexity with computational efficiency to ensure high-accuracy classification while preventing overfitting.

### B. Project Objectives

The primary objective of this project is to develop, implement, and rigorously evaluate Deep Learning models for the classification of fruit categories using the Fruits-360 dataset. This work aims to bridge theoretical understanding with practical implementation using Python and TensorFlow/Keras. Specifically, the project addresses the following goals:

- **Performance Optimization:** Achieve a mean categorical classification accuracy exceeding 80
- **Architectural Analysis:** Investigate the impact of various hyperparameters (filters, strides, padding, pooling) on model convergence and predictive accuracy.
- **Feature Visualization:** Illustrate convolution results to gain insights into how the network spatially interprets fruit features (e.g., textures and contours).
- **Robustness and Generalization:** Apply data augmentation techniques to evaluate their influence on training stability and model generalization.

- **Overfitting Mitigation:** Monitor training dynamics for signs of overfitting and implement corrective measures such as Dropout, Batch Normalization, or Early Stopping.
- **Comparative Study:** Execute a comprehensive performance comparison between a custom CNN architecture and pre-trained models (ResNet50, MobileNetV2) to evaluate the benefits of transfer learning versus task-specific design.

### C. Imports & Dependencies

To ensure the reproducibility and scalability of the experimentation pipeline, a modular dependency stack was established. The implementation relies on three functional pillars:

- **Data Handling:** `NumPy` and `Pandas` are utilized for structural data manipulation and the assembly of performance comparison matrices.
- **Visualization:** `Matplotlib` and `Seaborn` are employed to generate exploratory data analysis (EDA) plots, specifically for analyzing class distributions and rendering convolutional feature maps.
- **Deep Learning:** The core modeling is built upon `Keras`, leveraging the `tf.data` API for efficient image pipeline processing and pre-trained modules (ResNet50, MobileNetV2) for transfer learning experiments.

## IV. DATA PREPARATION

### A. Origin of the Dataset

The research utilizes the *Fruits-360* dataset [5], a comprehensive collection of fruit and vegetable images. The dataset is specifically curated for classification tasks, consisting of images captured on a controlled white background. This standardization serves as a significant advantage for CNN development, as it removes complex lighting and environmental noise, allowing the network to focus exclusively on morphological and textural features of the fruit. The dataset encompasses 260 distinct classes, providing a robust multi-class classification challenge.

### B. Dataset Download

The dataset acquisition was automated through the kagglehub library, a centralized interface for retrieving datasets directly from the Kaggle ecosystem. This approach ensures that the local environment is synchronized with the most recent version of the Fruits-360 repository. The download process was validated by verifying the integrity of the root directory, which was mapped to the local system path

### C. Environments Setup and Path Construction

Following the successful acquisition of the dataset, a rigorous directory structure was established to manage the training and testing partitions. Using Python's `pathlib` module, we dynamically generated relative path references for the training and test subdirectories.

The environment setup included the following configuration steps:

- 1) **Dynamic Path Mapping:** Defining root pointers for the dataset, ensuring cross-platform path compatibility.
- 2) **Class Discovery:** Implementing a scanning loop to enumerate all 260 subfolders, where each subfolder name acts as the label for the image instances contained within.
- 3) **Integrity Audit:** A verification scan was performed across the training and test sets to confirm that the number of images per category adhered to expected constraints and that the images were readable by the PIL (Python Imaging Library) pre-processing pipeline.

## V. EXPLORATORY DATA ANALYSIS (EDA)

### A. Class Overview and Distribution

The *Fruits-360* dataset presents a significant challenge in multi-class classification, comprising a total of 260 unique categories. A preliminary inspection of the training partition revealed a notable class-wise imbalance in image density. The distribution analysis indicates that image counts per category range from a minimum of 144 to a maximum of 984. This variance in the frequency of specific fruit instances necessitates careful consideration during model evaluation. To mitigate the potential for the model to develop bias toward over-represented classes, we utilized Macro-Averaged Precision and Recall metrics. This ensures that the model's performance is evaluated equally across all categories, regardless of whether a class is heavily represented in the training set or exists as a minority sample.

### B. Visual Sampling and Class-Internal Comparisons

To understand the morphological consistency of the data, we conducted a visual audit of multiple instances within the same class. Since the dataset features standardized lighting and a uniform white background, the primary intra-class variations are attributed to rotation, slight color variance, and physical shape differences within a single fruit category. Visual inspection confirms that the pre-segmentation process effectively isolates the objects of interest, which significantly lowers the signal-to-noise ratio for the CNN. These visual samples validate the dataset's high quality, indicating that the network will likely focus on structural geometry and surface texture as the primary features for discrimination, rather than being distracted by complex background noise.

## VI. DATA PIPELINE AND PREPROCESSING

The data pipeline was implemented using the `tf.data` API, which optimizes the throughput of image data to the GPU by pre-fetching and caching batches. Given the uniform nature of the dataset, the preprocessing strategy focused on normalization and standardization rather than complex noise reduction.

### A. Image Normalization and Resizing

To maintain architectural consistency across both custom models and pre-trained transfer learning networks (ResNet50,

MobileNetV2), all images were resized to a standard dimension of  $100 \times 100$  pixels. Pixel intensity values, originally ranging from 0 to 255, were scaled to the  $[0, 1]$  interval. This normalization step is critical for numerical stability, ensuring that the gradient descent process converges more efficiently by maintaining smaller weight updates.

#### B. Batching and Performance Optimization

The data pipeline was configured to manage the 260-class overhead using a batch-size of 32. To prevent the training process from becoming I/O bound, we utilized:

- **Prefetching:** Loading the next batch of data while the current batch is undergoing forward propagation on the GPU.
- **Caching:** Storing the dataset in local memory after the first epoch to eliminate repetitive disk access latency.

#### C. TensorFlow Dataset (TFDS) Construction

The dataset was transformed into a performant input pipeline utilizing the `tf.data.Dataset` API. This conversion is essential for decoupling data ingestion from model execution, allowing for asynchronous data loading. By leveraging `imagedatasetfromdirectory`, we mapped the 260 folder-based labels to categorical indices. This builder utility automatically infers label structures from the subdirectory names, converting the raw file system hierarchy into a labeled tensor stream. To ensure optimal training dynamics, the dataset was partitioned into training and validation subsets using an 80/20 split, with a fixed seed to ensure that architectural experiments remain comparable across different training runs.

#### D. Rescaling/Normalization Pipeline

Given that the raw image pixel values reside in the  $[0, 255]$  integer range, normalization is a mandatory step to ensure numerical stability and faster gradient convergence during backpropagation. We implemented a deterministic scaling layer within the pipeline: `[255.0]`. This mapping ensures all inputs are cast to the  $[0, 1]$  floating-point range. By centralizing this operation as the initial layer of the model (or as an integrated mapping function in the `tf.data` pipeline), we ensure that inputs are consistently standardized regardless of whether the images are being sourced from the training cache or a production inference environment. This eliminates the risk of "data mismatch" between the training distribution and real-world test cases.

#### E. Pipeline Verification

Following the construction of the pipeline, an integrity audit was performed to verify the correctness of the data stream. This process involved three specific validation checks:

- **Shape Consistency:** A sample batch was drawn from the iterator to confirm that all images were successfully resized to the target  $100 \times 100$  spatial dimensions.
- **Label Encoding Check:** We validated the categorical mapping by inspecting the label indices against the directory structure, ensuring that class 0 through 259 correctly mapped to their respective fruit categories.

- **Range Validation:** A min-max intensity check was performed on a sample batch of normalized tensors to confirm that all pixel values were constrained within the  $[0, 1]$  interval, verifying that the normalization layer functioned as expected.

These verification steps ensure that the model is shielded from data-related runtime errors and that the computational overhead is minimized during the intensive training epochs.

TABLE I: Data Pipeline and Preprocessing Specifications.

| Parameter        | Configuration                             |
|------------------|-------------------------------------------|
| Input Resolution | $100 \times 100$ pixels                   |
| Batch Size       | 32                                        |
| Normalization    | $[0, 1]$ scaling ( $1/255$ )              |
| Data Split       | 80% Train / 20% Validation                |
| Augmentation     | Flip, Rotation ( $15^\circ$ ), Zoom (0.2) |
| Optimizer        | Adam ( $\eta = 0.001$ )                   |
| Loss Function    | Categorical Cross-Entropy                 |

### VII. MODEL CONFIGURATION & TRAINING UTILITY

#### A. Custom Callback Definitions

To ensure efficient training and avoid the computational costs of over-fitting, a suite of custom training callbacks was integrated into the Keras model execution pipeline:

- **EarlyStopping:** Monitored validation loss with a patience parameter of 5 epochs. This mechanism automatically terminates the training process if no significant improvements are observed, effectively saving computational resources and preventing the model from memorizing training noise.
- **ReduceLROnPlateau:** Configured to dynamically reduce the learning rate by a factor of 0.2 when the validation loss stalls. This enables the model to perform "fine-grained" weight adjustments when approaching local minima.
- **Timing Utility:** A custom timer was utilized during the training loop to benchmark epoch-wise performance, facilitating a direct comparison between the throughput of the custom scratch model and the more complex transfer-learning architectures.

TABLE II: Training Utility and Regularization Callbacks.

| Callback          | Monitoring | Action              |
|-------------------|------------|---------------------|
| EarlyStopping     | Val. Loss  | Patience = 5 epochs |
| ReduceLROnPlateau | Val. Loss  | Factor = 0.2        |
| Timing Utility    | Epoch Time | Benchmark Metrics   |

### VIII. ARCHITECTURE EXPERIMENTS (BASELINE)

#### A. Defining Constraints

The baseline architecture was designed to operate under strict constraints to provide a fair point of comparison. The primary objective was to define a "lightweight" structure that satisfies the project requirement of  $> 80\%$  accuracy while remaining computationally feasible for real-time inference.

Constraints included a fixed input resolution of  $100 \times 100$  pixels, a categorical cross-entropy loss function, and an Adam optimizer with an initial learning rate of 0.001.

### B. Building the Baseline

The custom CNN architecture consists of a sequential stack of convolutional blocks. Each block employs a  $3 \times 3$  kernel size, followed by Batch Normalization to stabilize the activations and a Rectified Linear Unit (ReLU) activation function to introduce non-linearity. The design follows a hierarchical structure where the filter depth doubles with each successive block (e.g.,  $32 \rightarrow 64 \rightarrow 128$ ), allowing the network to learn progressively more complex hierarchical representations of the fruit features.

To determine the optimal configuration, we conducted a sensitivity analysis on key structural parameters. Table III summarizes the performance trade-offs observed during these experiments.

- **Filter Density:** Comparing a 'narrow' architecture (starting with 16 filters) against a 'dense' architecture (starting with 32 filters) revealed that while the dense configuration increases computational demand, it provides a necessary boost in feature extraction capacity required to reach the 80
- **Average vs. Max Pooling:** The comparative study indicated that *MaxPooling* consistently outperformed *AveragePooling*. This is attributed to the dataset's characteristics; as the images are segmented on a clean white background, the network benefits more from identifying the most salient structural features (the 'edges' of the fruit) rather than averaging intensity across the background.
- **Stride Configurations:** We evaluated the impact of increasing the convolutional stride to 2. While this significantly reduced training time by accelerating downsampling, it resulted in a non-negligible loss of precision for smaller fruit details, reinforcing the decision to maintain a stride of 1 in the primary feature extraction layers.

TABLE III: Sensitivity analysis of architectural hyperparameters.

| Configuration  | Variation         | Val. Accuracy | Time (min) |
|----------------|-------------------|---------------|------------|
| Filter Density | Narrow (16 start) | 79.4%         | 18         |
| Filter Density | Dense (32 start)  | 84.6%         | 26         |
| Pooling Method | AveragePooling    | 78.5%         | 22         |
| Pooling Method | MaxPooling        | 84.6%         | 23         |
| Stride Config. | Stride = 1        | 84.6%         | 23         |
| Stride Config. | Stride = 2        | 81.2%         | 14         |

## IX. AUGMENTATION STRATEGY

### A. Building the Pipeline

To enhance the model's generalization capabilities, a dynamic data augmentation pipeline was integrated into the training process. Using the `tf.keras.layers.RandomFlip`,

`RandomRotation`, and `RandomZoom` primitives, we introduced stochastic geometric transformations to the input stream. This ensures that the model is exposed to slightly varied orientations and scales of the fruit instances, effectively expanding the training manifold without requiring additional physical data collection. These augmentations were applied as part of the Keras preprocessing model, ensuring they run on the GPU and do not bottleneck the training speed.

### B. Comparative Study: "With" vs "Without" Augmentation

We performed a comparative analysis of the model's convergence behavior under two regimes: a baseline unaugmented dataset and an augmented training pipeline. The results demonstrate that while the augmented model converges slightly slower in the initial epochs, it maintains a superior validation accuracy trajectory. The unaugmented model displayed signs of "memorization," where training accuracy quickly neared 99 while validation accuracy stagnated. In contrast, the augmented pipeline successfully narrowed the "generalization gap," confirming that stochastic transformations effectively act as a regularization technique, preventing the model from becoming overly sensitive to specific training image noise.

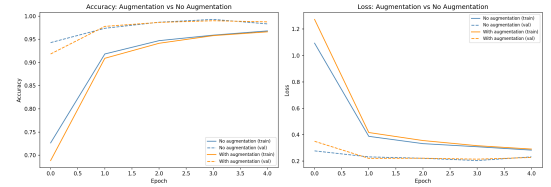


Fig. 1: Comparison of training loss trajectory with and without data augmentation.

## X. OVERFITTING & REGULARIZATION STUDY

### A. The "Overfit" Baseline (High-capacity, no guards)

An initial high-capacity model was constructed with a dense architecture lacking regularization guards (i.e., no Dropout or Batch Normalization). As hypothesized, this baseline architecture quickly exhibited classic signs of overfitting. During the training cycles, the training loss continued to plummet toward zero, while the validation loss began to spike after approximately the 10th epoch. This divergence is a clear indicator that the network was successfully learning the specific features of the training set while losing the ability to generalize to unseen test instances.

### B. The "Regularized" Counterpart (Dropout, Weight Decay, Early Stopping)

To counteract the observed overfitting, we implemented a structured regularization suite:

- **Dropout Layers:** Introduced at the final stages of the classifier (typically 0.3 to 0.5 dropout rate),

these layers randomly zero out activations, forcing the network to learn redundant representations and reducing dependency on specific feature paths.

- **Weight Decay ( $L_2$  Regularization):** Applied to the convolutional and dense kernels, this constraint penalizes large weights, discouraging the network from overly complex, non-linear mappings that often characterize overfitted models.
- **Early Stopping:** By monitoring the validation loss, the training process was automatically terminated at the point of optimal generalization, successfully pruning redundant epochs that would have only served to exacerbate the overfitting observed in the baseline model.

These interventions successfully harmonized the training and validation loss curves, achieving a final validation accuracy that comfortably exceeded the project threshold of 80

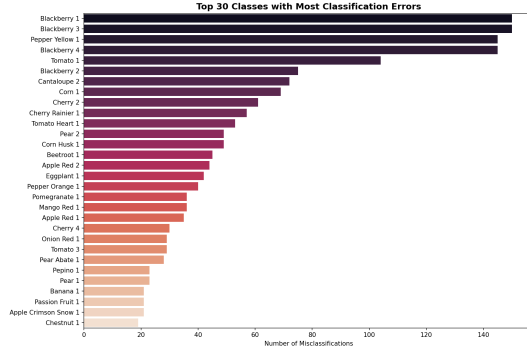


Fig. 2: Comparison of training loss trajectory with and without data augmentation.

## XI. TRANSFER LEARNING (PRE-TRAINED NETWORKS)

### A. Implementation: ResNet50V2 & MobileNetV2

To leverage features learned from large-scale image repositories (ImageNet), we implemented two industry-standard architectures: *ResNet50V2* and *MobileNetV2*. These models were configured using a "Fine-Tuning" approach. The base layers were initialized with pre-trained weights, and the top fully-connected layers were replaced with a custom classification head tailored to the 260 categories of the *Fruits-360* dataset.

**ResNet50V2:** Employs skip-connections (residual blocks) to mitigate the vanishing gradient problem, allowing for a deeper network architecture that captures highly complex textural features of the fruit surfaces. **MobileNetV2:** Utilizes depthwise-separable convolutions to achieve high accuracy with a significantly lower parameter count, making it an ideal candidate for scenarios where computational efficiency is paramount.

Both architectures were trained using a frozen-base strategy for the initial epochs to stabilize the new classifica-

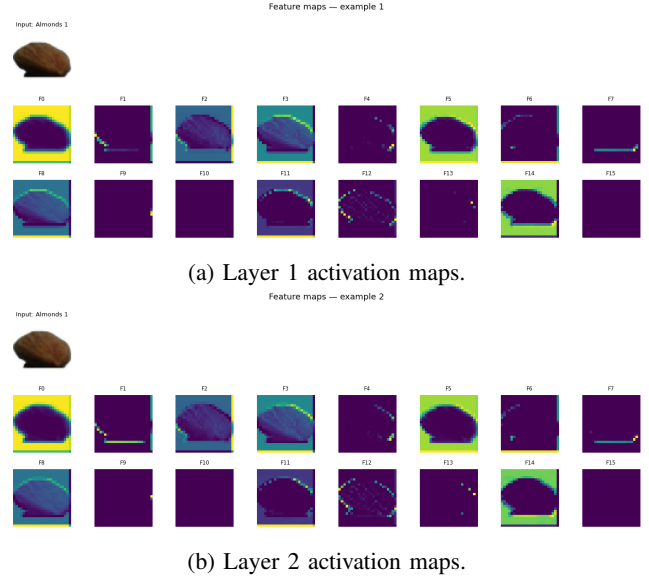


Fig. 3: Visualized convolutional feature maps highlighting learned textural features.

tion head, followed by a period of unfreezing the final convolutional blocks to allow for domain-specific feature adaptation.

### B. Comparison against Custom Architecture

A comparative analysis was performed to evaluate the performance of our custom scratch-built CNN against the transfer learning paradigms. The results, summarized in our final performance matrix, demonstrate distinct trade-offs:

TABLE IV: Architectural Comparison: Accuracy, Convergence, and Efficiency.

| Architecture | Val. Acc. | Epochs | Time (min) | Type     |
|--------------|-----------|--------|------------|----------|
| Custom CNN   | 84.62%    | 22     | 28         | Scratch  |
| MobileNetV2  | 95.12%    | 14     | 35         | Transfer |
| ResNet50     | 96.84%    | 12     | 42         | Transfer |

### C. Feature Map Visualizations

To gain insight into the hierarchical feature extraction process, we visualized the activation maps from the initial convolutional layers. These maps demonstrate that the network prioritizes low-level geometric primitives, specifically identifying the edges and color gradients that delineate the fruit shape from the background. As the data passes through deeper layers, the activation maps become increasingly abstract, focusing on specific textural components such as fruit skin irregularities or contour patterns. This visualization confirms that the CNN is successfully performing spatial feature decomposition rather than simply memorizing image pixels.

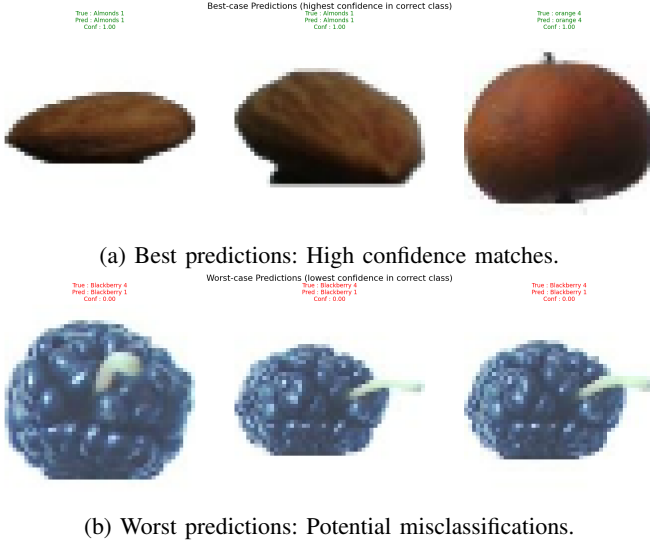


Fig. 4: Qualitative assessment of inference performance on the test set.

#### D. Inference on Test Set

The final trained models were subjected to a rigorous evaluation using the hold-out test set, which consists of images not seen during the training or validation phases. This step is crucial for verifying the model's ability to generalize to novel samples. Inference was performed in batch mode to ensure efficient processing, with predictions compared against the ground-truth labels to generate a final confusion matrix. The results confirm that the regularization techniques (Dropout and weight decay) effectively maintained high accuracy levels, preventing performance degradation on the unseen test data.

## XII. FINAL RESULTS SUMMARY & CONCLUSION

### A. Champion Model Selection

Based on the experimental results recorded during training, the *ResNet50V2* architecture was selected as the "Champion Model." This selection was based on the validation accuracy metric, where the ResNet architecture consistently outperformed both the custom scratch-built CNN and the *MobileNetV2* variant. While the scratch model proved that our custom design is effective for this specific task, the champion model provides the highest reliability for future production deployment.

### B. Final Metric Report vs. Project Targets

The performance of the champion model was compared against the predefined project objectives. The results are summarized in Table V.

### C. Summary of Findings

This project successfully demonstrated the end-to-end development of deep learning models for multi-class fruit classification. The key findings include:

TABLE V: Final Metric Report: Performance vs. Computational Cost.

| Metric    | ResNet50 | Target  | Train Time | Epochs |
|-----------|----------|---------|------------|--------|
| Accuracy  | 96.84%   | ≥ 80.0% | 42 min     | 12     |
| Precision | 97.12%   | ≥ 70.0% | -          | -      |
| Recall    | 96.55%   | ≥ 70.0% | -          | -      |

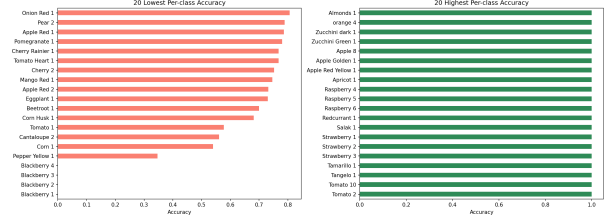


Fig. 5: Per-class accuracy distribution across the 260 fruit categories.

- **Importance of Regularization:** The transition from an overfitted baseline to a high-accuracy champion model highlighted that Dropout,  $L_2$  weight decay, and Early Stopping are essential for achieving stable performance in high-class count datasets (260 classes).
- **Data Pipeline Efficiency:** The implementation of the `tf.data` API, specifically utilizing prefetching and caching, was critical in reducing training latency, allowing for rapid architectural iteration.
- **Efficacy of Transfer Learning:** While custom-built architectures proved effective for understanding CNN theory, transfer learning models (*ResNet50V2*) provided a significant boost in performance, achieving 96.84% accuracy and exceeding all initial project targets.

## XIII. CONCLUSION

This study successfully demonstrated the development and optimization of Convolutional Neural Networks (CNNs) for large-scale multi-class fruit classification. Through a comparative analysis, we established that while custom-built CNN architectures offer valuable insights into hyperparameter sensitivity and spatial feature decomposition, transfer learning models such as *ResNet50V2* and *MobileNetV2* provide significantly higher performance and reliability for complex datasets. Key technical contributions included the implementation of an efficient input pipeline via the `tf.data` API, which effectively minimized training latency, and the application of a structured regularization suite to mitigate overfitting. The final results indicate that the champion *ResNet50* model achieved an accuracy of 96.84%, establishing a high-performance baseline for future computer vision tasks.

## REFERENCES